Artificial intelligence and software design

Paolo Burgio
paolo.burgio@unimore.it

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA
High Performance Real Time Lab

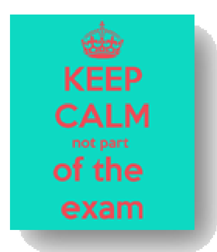
I TUOI
FUTURI MEDICI
STANNO USANDO
CHATGPT PER
SUPERARE GLI
ESAMI DI MEDICINA

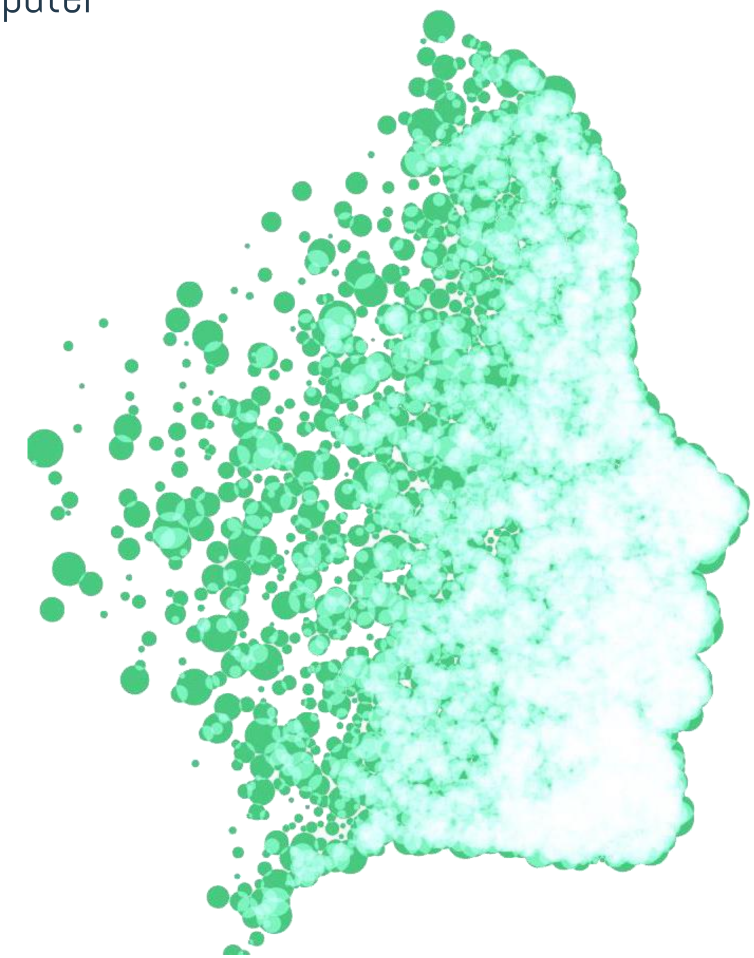
AI– A brief recap

Instead of “giving orders” to the computer about what to do (traditional informatics, based on algorithms)...

- Reproduce (and improve!) a brain inside a computer
- Train it, and it learns
- Not anymore deterministic

AI is taught later in MS, here I will explain some basic concepts that might be helpful to you in your career as dev





(My) open questions

Where to apply AI?

- Single component vs. e2e

Use AI for code dev

- You teach me than more I can teach you
- Vibe coding? Or co-pilots? (Implications on the job model)

Can we structure the SW development process to enable scalability?

- Vibe coding?
- Enforce design patterns? What about SOLID and CLEAN?
- Use AI in design? And testing?

Pros and cons

Pros (countless)

- Increase productivity
- Automate not only code dev, but all processes
- ...

Cons

- Not deterministic
- Require datasets
- Replaces workforce (that cannot work without AI)
- ...

The DORA report from Google *

"In 2025, the central question for technology leaders is no longe

if they should adopt AI, but how to realize its value."

Not anymore a novel disruptive technology, it's a reality

- ..and it's here to stay!

I am not saying you can't do research on AI..

- ..what I am saying is that, once the hype effect is gone, we need to approach AI in a **systematic and structured** manner

** more than 100 hours of qualitative data and survey responses from 5,000+ professionals from around the world*

DORA's main outcome

"AI's primal role in SW development is the one of an amplifier"

› It magnifies the strengths and dysfunctions of your organization/team

The **returns on AI investment** comes from the underlying organizational system

› The *quality* of the internal platform

› The clarity of *workflows*

› The alignment of teams

The risk

› Creating localized pockets of productivity that are often lost to downstream chaos

DORA report - Key aspects

AI is pervasive

- 90% of respondents use AI
- 80% declare it increases productivity
- 30% do not trust results from AI

You shall not simply adopt tools

- It's about practices, and workflows
- (It's about having a plan, and not having the AI drive you)

AI improves SW delivery throughput and pace

- But it also increases SW instability
- Teams and single devs are using it effectively for speed
- Orgs and their processes cannot keep the pace

DORA report - Key aspects (cont'd)

Value Stream Management (VSM) as an effective practice (“force multiplier”)

- Continuous monitoring and improving the workflow from idea/design to to customer
- Suitable for Agile methodologies, with fast Time-To-Market (hence, AI)
- Translates team productivity into **measurable** improvements

Identified seven type of team profiles

- We'll see this later

Monitoring is key to avoid disasters

- Need to identify metrics and KPIs referring to the process, not to the technology!

AI for coding

..you teach me!

Question: How do you use AI?

..you teach me!

Question: How do you use AI?

I use it for

- Prototyping and scaffolding (for instance, I am not good at all at UX programming..)
- Debugging: error detection & prediction
- Support for punctual problems
  - *"How do I configure an Apache server to reject all GET requests?*
  - *"Can you scaffold a Visitor pattern with 4 classes and 30 visitors, with all interfaces?"*

Don't confuse the two things!

Increase one's productivity **VS.** "quick & dirty" *I-don't-know-what-I'm-doing* approach

(The latter is ok for non-experts in the field, who need to do prototyping)

Copilot* vs Vibe coding

Besides prototyping... how can you effectively build code using AI assistants?

- Answer: this is part of your “freedom” so, it’s up to you
- ..if you still are in the “frame” of **good design, good processes** (, ethics) and company policies

Business models for LLMs – from assistants to AI Agents

- ChatGpt – Generic but mainly B2C
- MS Copilot – B2C
- Claude – B2B

Paid vs. free

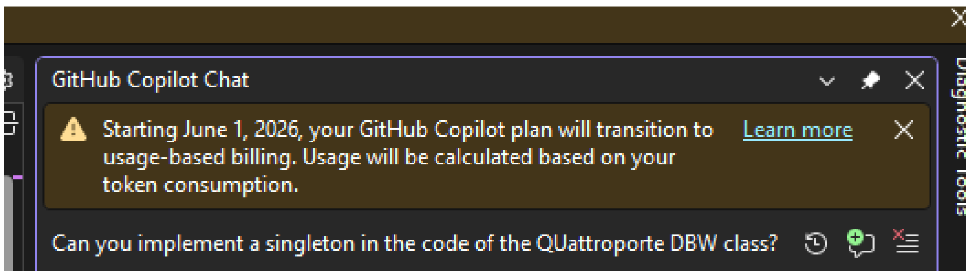
- Basically, better models
- And training vs. no training - Your company shall not share code, nor expertise, nor data

**Disclaimer: this is not about Microsoft Copilot ☺*

Only one hint for you...

Let the AI be a tool/friend,

not your replacement



- Mi dica cosa sa fare che non possa fare con ChatGPT

How?

There is still something that AI needs... and will always need

A context

- Domain information “context”
- Goals
- Ground (dataset)

AI Agents

Beyond “simple” helpers, they can accomplish complex tasks

- Up to a full coding + testing + delivery pipeline

Key features

- Still, they need you to define the context and goal
  - the agent decides the sequence of actions needed to reach it
- Easy integration with external software
  - Databases Svc APIs,business applications, other agents
- Planning & Memory
  - They self-manage the process

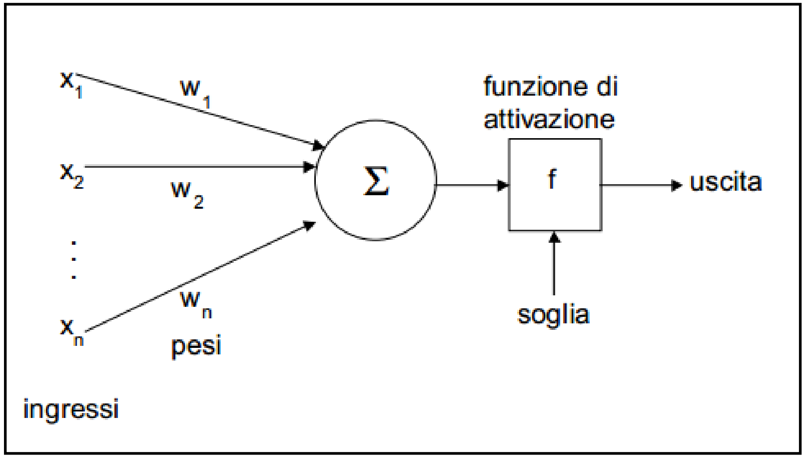
Best for

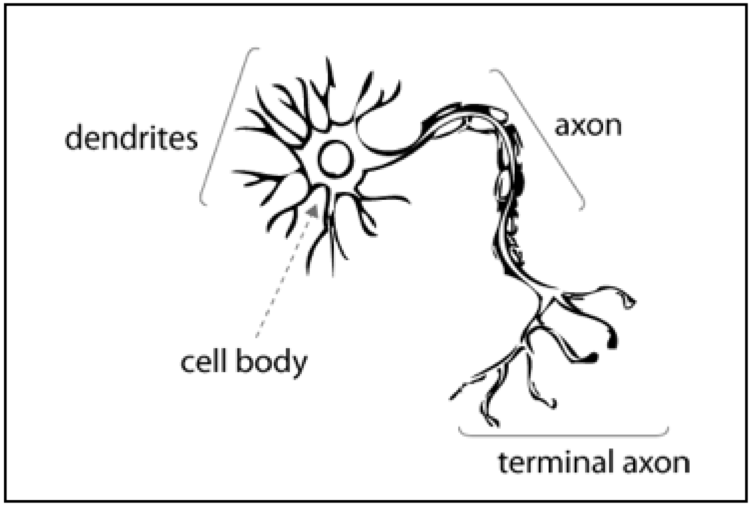
- Customer Support & troubleshooting large systems
- Business Operations and systems integration by automating data workflows

Designing AI systems

Convolutional Neural networks

- Bio-inspired
- Based on neurons arranged in layers
  - And sub-layers
- Convolutional neural network
  - Perform Convolutions

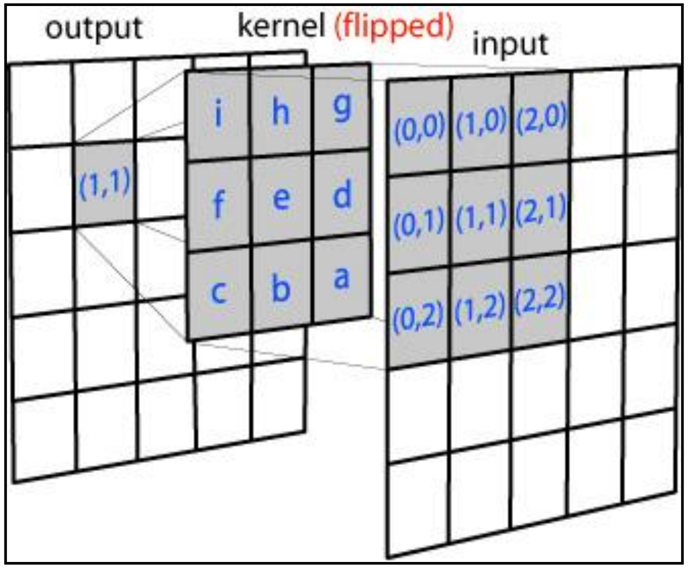




Convolution

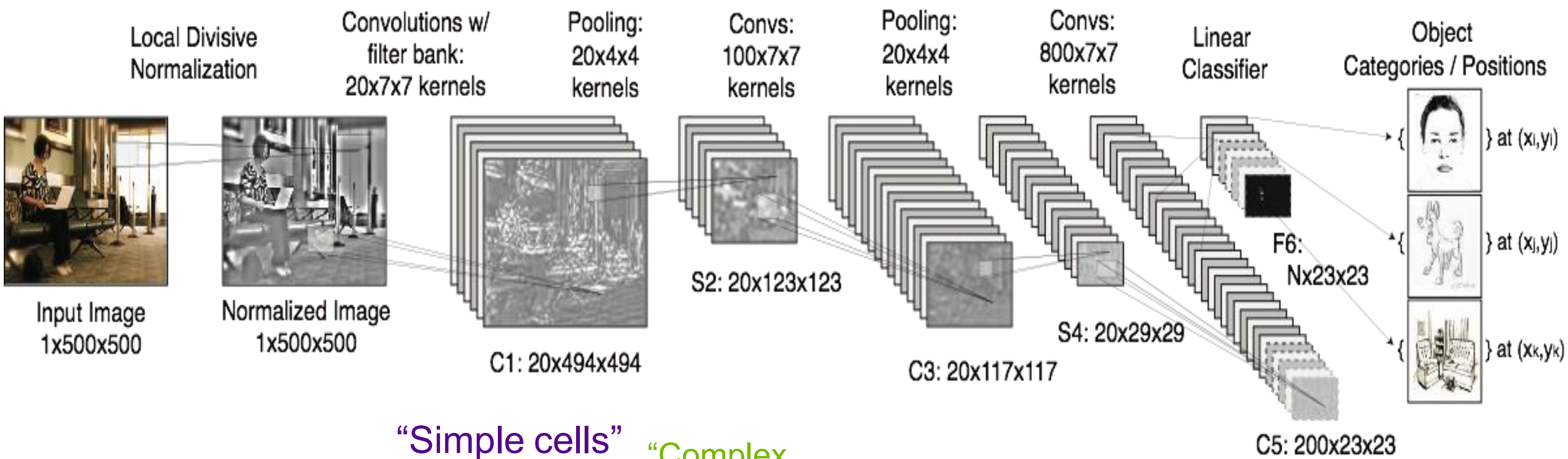
- Computation-intensive
- Suitable for implementation in hardware
- In computer vision, blurring

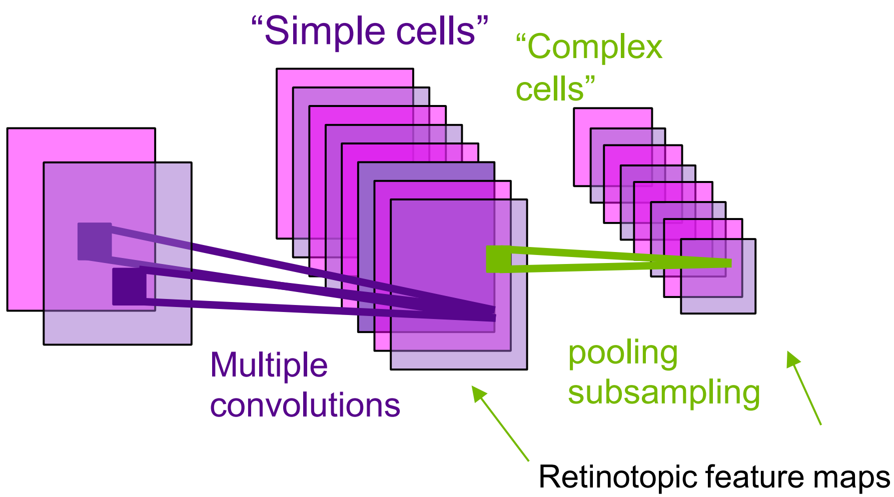
$$(f * g)(t) \stackrel{\mathrm{def}}{=} \int_{-\infty}^{\infty} f(\tau) g(t-\tau)\, d\tau = \int_{-\infty}^{\infty} f(t-\tau) g(\tau)\, d\tau.$$



The convolutional net model

› (Multistage Hubel-Wiesel system)





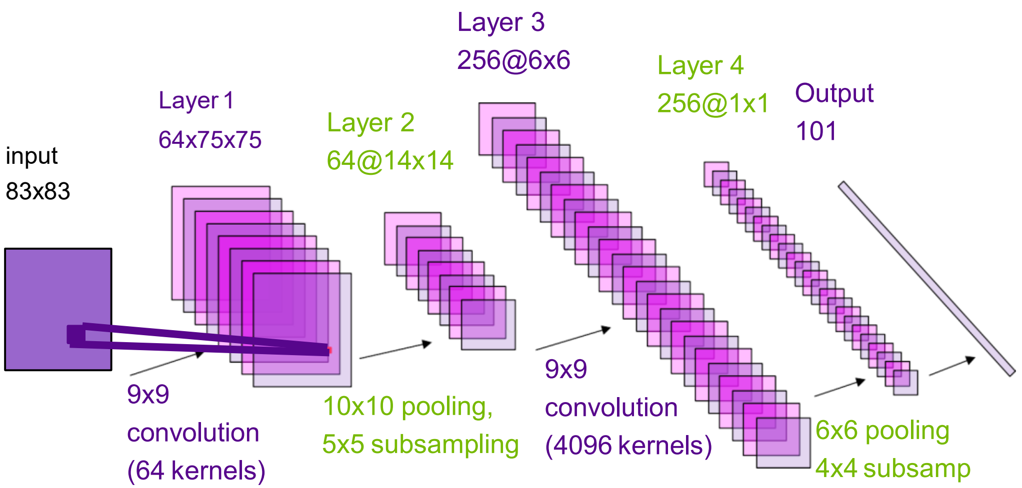
Challenge 1 – building the network

Network topology

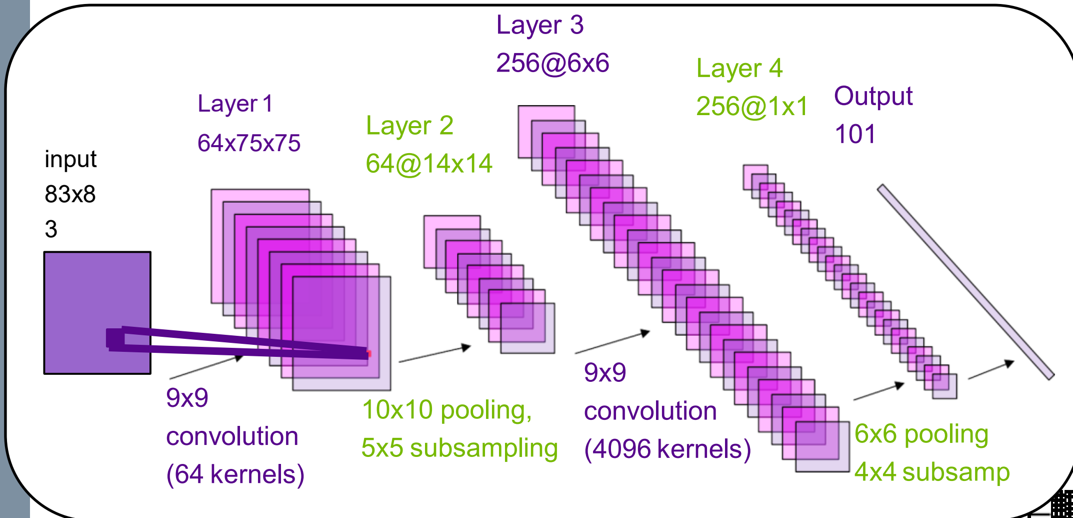
- How many layers and sublayers?
- How big they are?
- How are they connected?

Neuron type

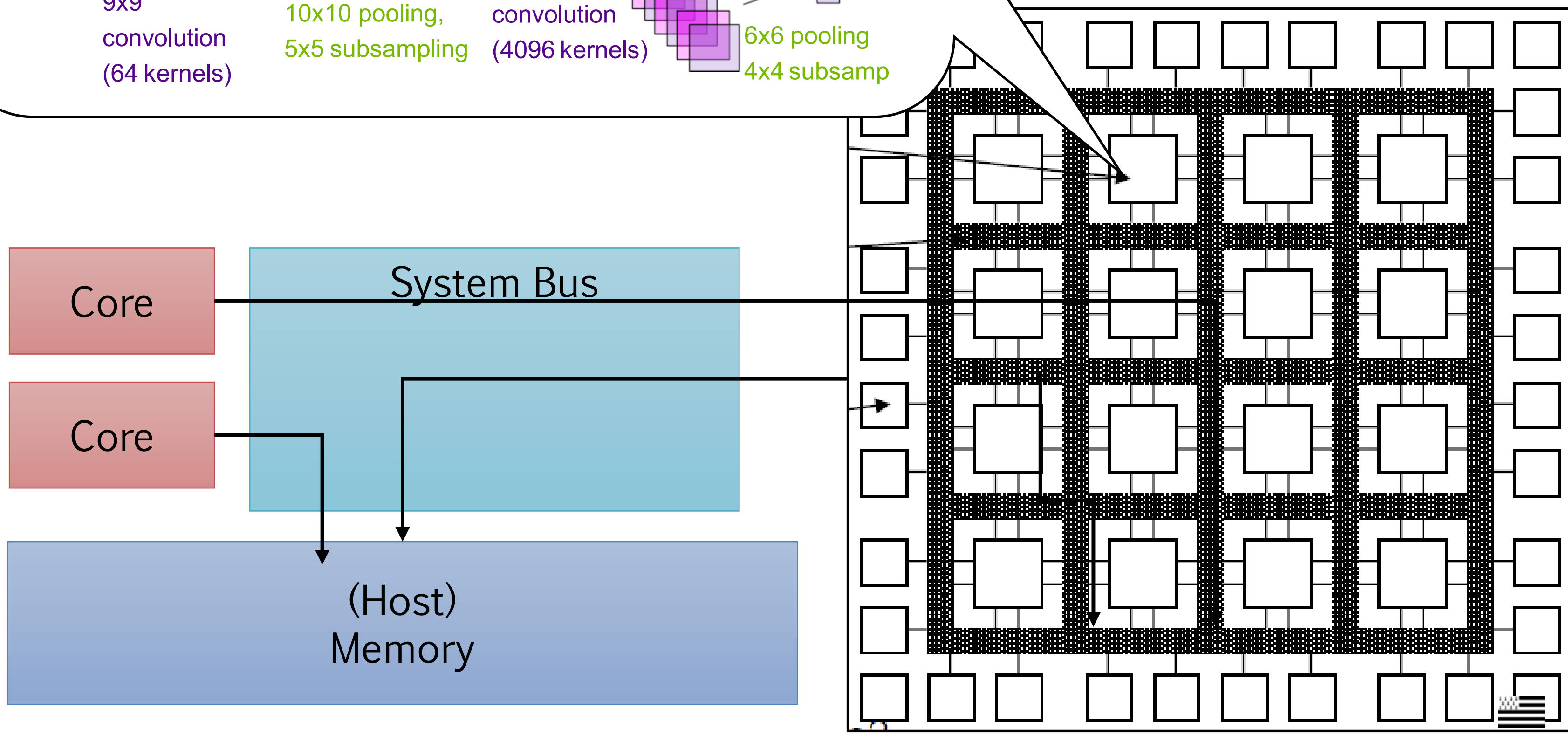
- CNN
- int/float datatypes
- How to perform pooling?



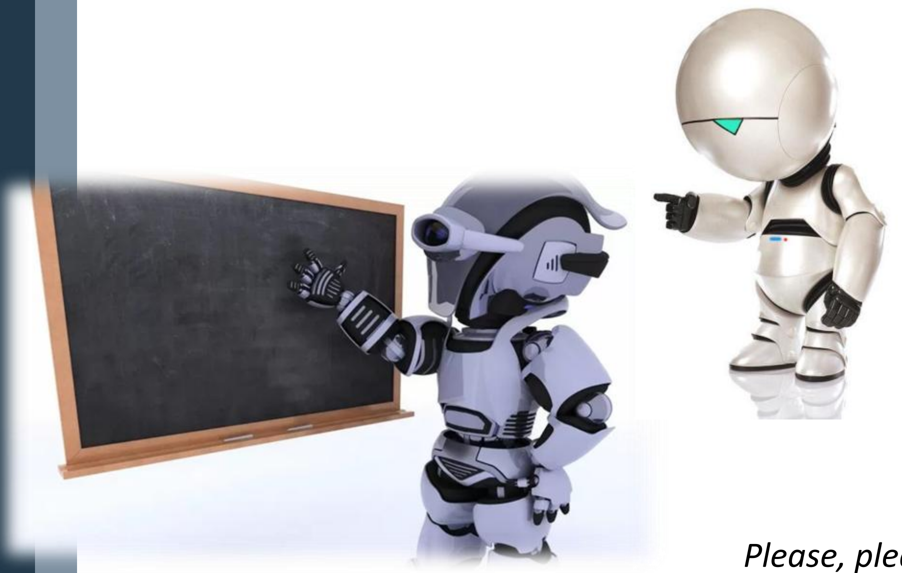
Challenge 2 – which computer?



- Here, an embedded board, for inference



Machines go to school...

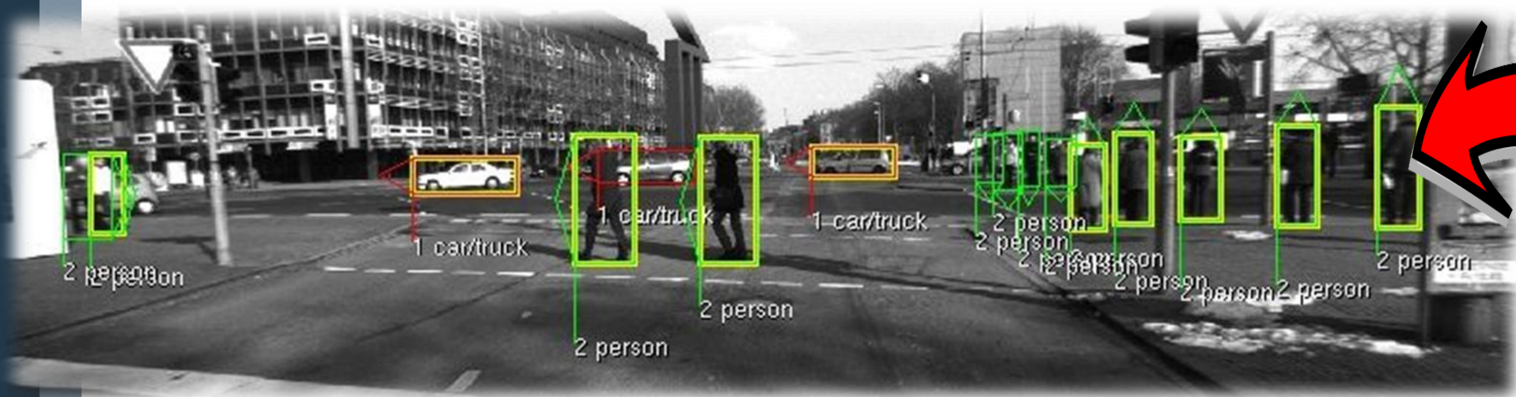


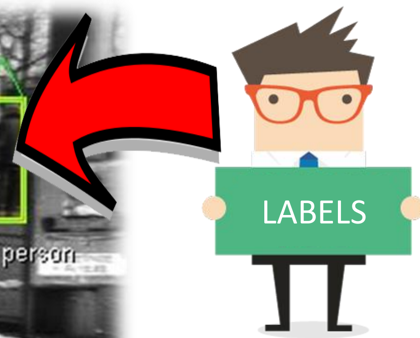
...machines learn!

- Near-human performance
- Surely, better than other SW artifacts
- There is no other (efficient way)

*Please, please, **please**, leave Skynet out of the discussion, ok?*

Challenge 3 - Training sets





We need real data!

- Why do you think Google does self-driving cars?
- Why do you think big cloud players want our data?
- <u>Collect, store and manage</u> data in a safe and GDPR-compliant manner




How to design a dataset?

Two approaches, not exclusive

- Model centric (Focus on the network model) vs. data centric (create/improve the dataset)

What you need, first

- A comprehensive view of how your model works
- The computational requirements of your use case (are we running on servers? But then we need to transmit the data!)
- SLA / timing requirements (is the user interaction async or sync?)

Key concepts to dataset design

Data quality

- Data must be relevant to the context, consistent (no duplicates..), and compliant to privacy, law, ethics

Data quantity

- Enough, but too much can lead to biases

Data coverage

- Shall cover all the requirements of your system
- (e.g., we plan to deploy a system that predicts traffic flows in Italy, but not in China)

Optimization

- For the accuracy, for the target computing platform..

Example: data coverage in Llama 3

	Pre Training	Supervised FineTuning	Preference FineTuning
General Knowledge	50%	52.66%	82%
Math and Reasoning	25%	21.19%	6%
Coding	17%	15%	7%
Multilingual	8%	3%	5%
Exam like	X	8%	x
Long Context	X	0.1	X

Source: https://arxiv.org/abs/2407.21783

Data lifecycle: an iterative process

Improving your data

Data Augmentation

- Creating variations of existing data to improve robustness and reduce bias.
- “Paolo is drinking water” – “Mario is drinking water”

Data Synthesis

- Generating new data, by randomly modifying existing data
- Example: random pedestrian generation in for autonomous parking scenario (Bachelor Thesis, y2018)
- Disclaimer. Random is usually pseudo-random! You insert this in the model

Human-in-the-Loop (HITL)

Def: "A model requiring human interaction"

- Typically, during a modeling & simulation process
- Used in testing to detect corner cases, and to test the robustness to unpredictable cases

In Machine learning

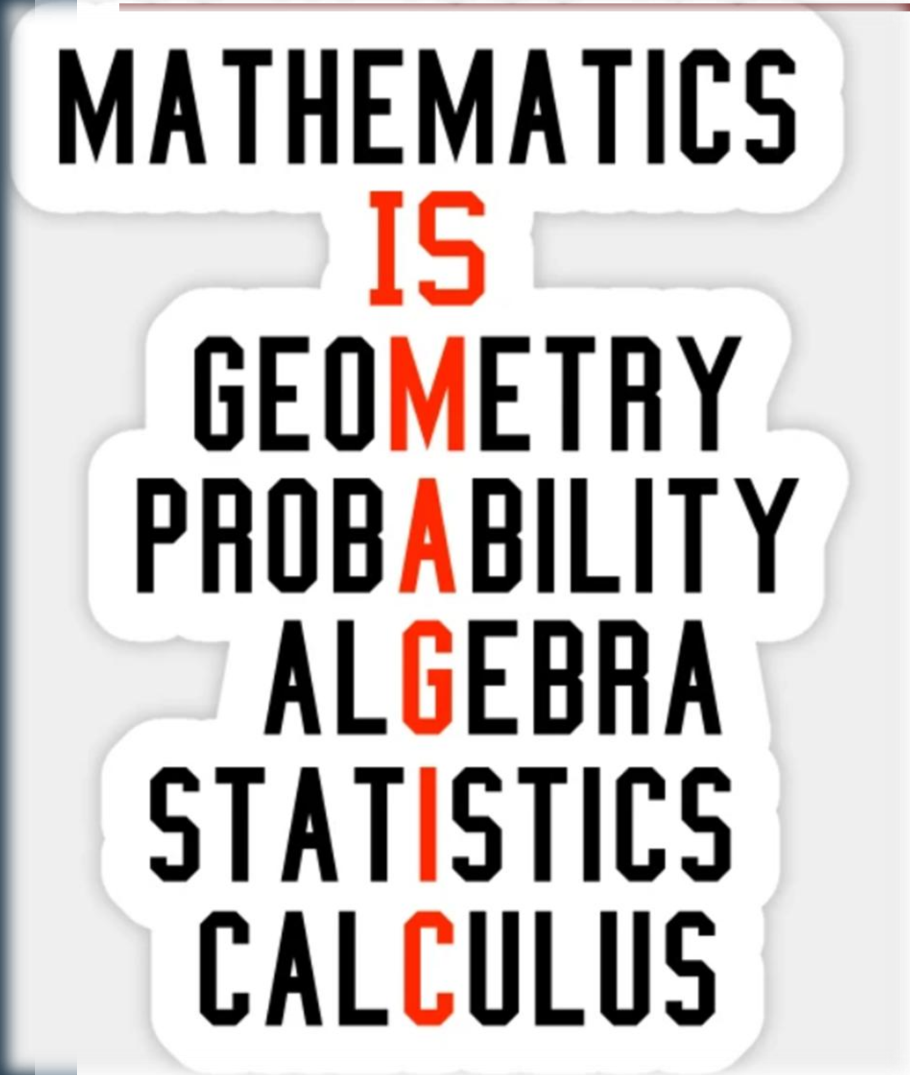
- During the building/training phases of a model
- Humans help selecting the most critical data, and corner cases

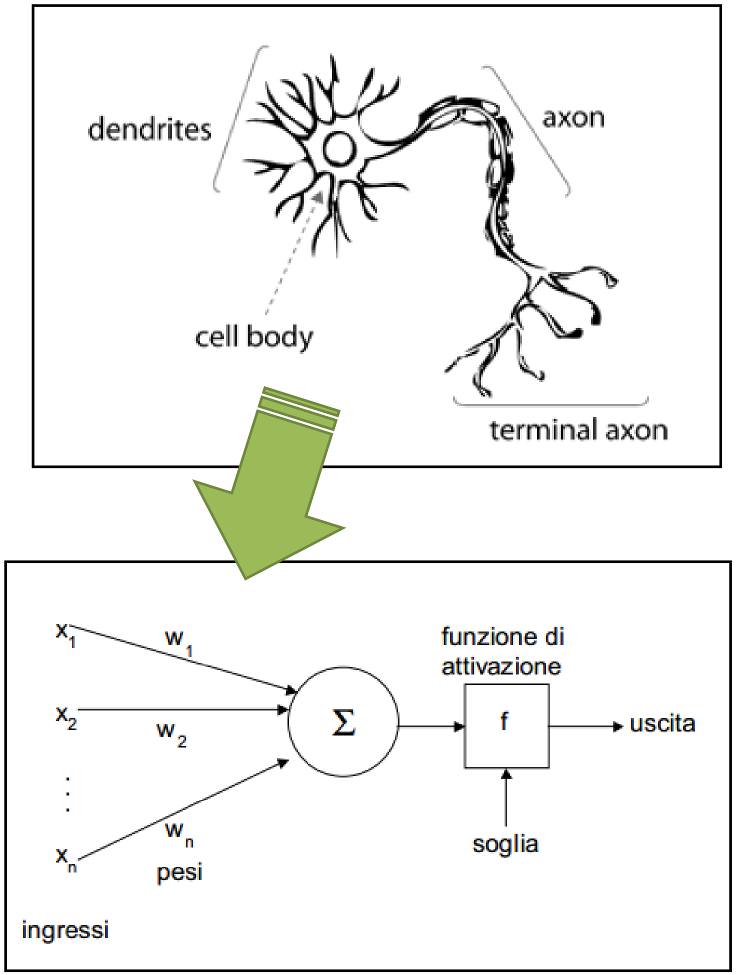
Pros/cons

- Performs better than random techniques (unlikes our previous example ☹)
- Requires time and effort

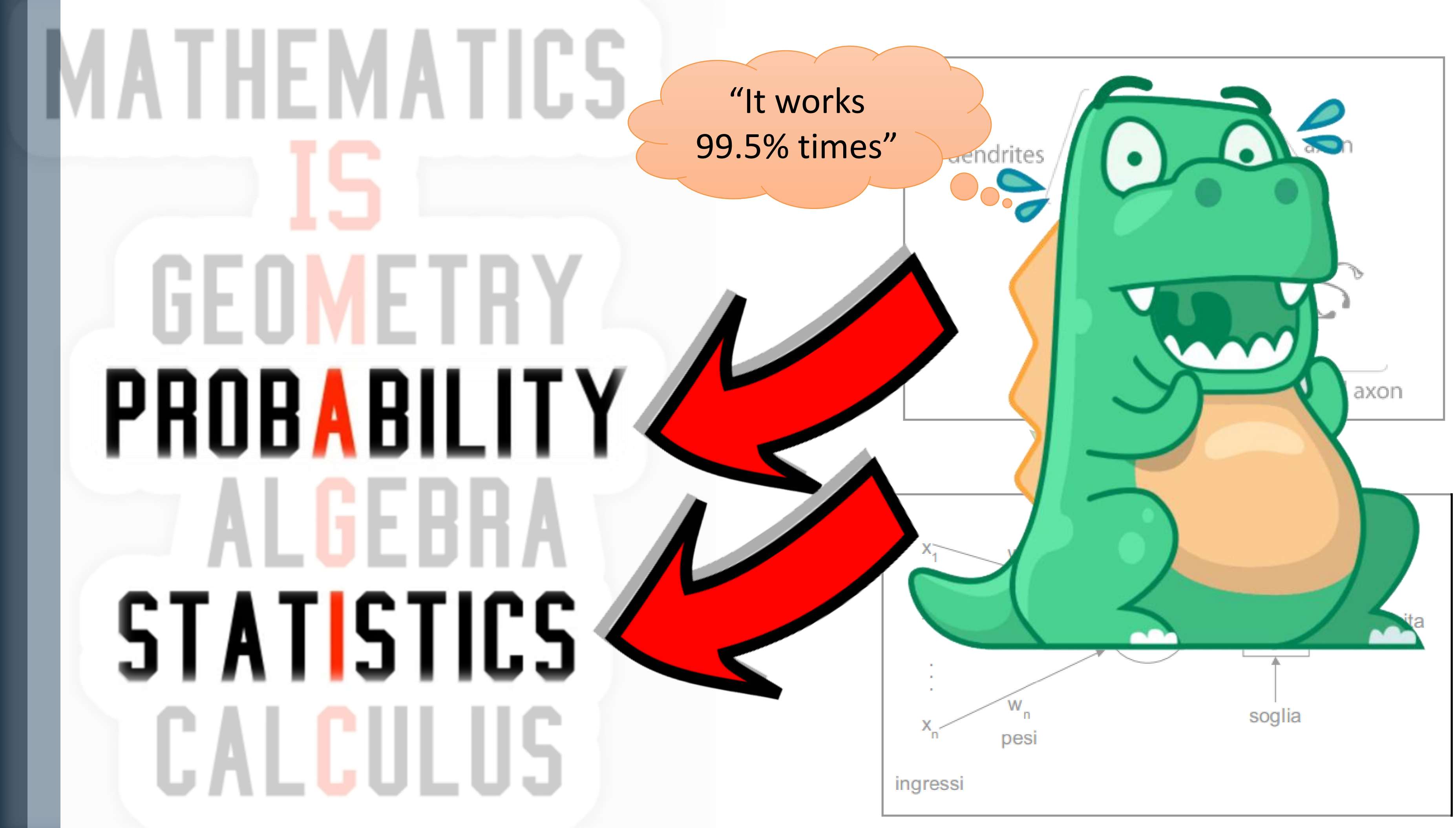
Managing uncertainty

It's statistics, baby!





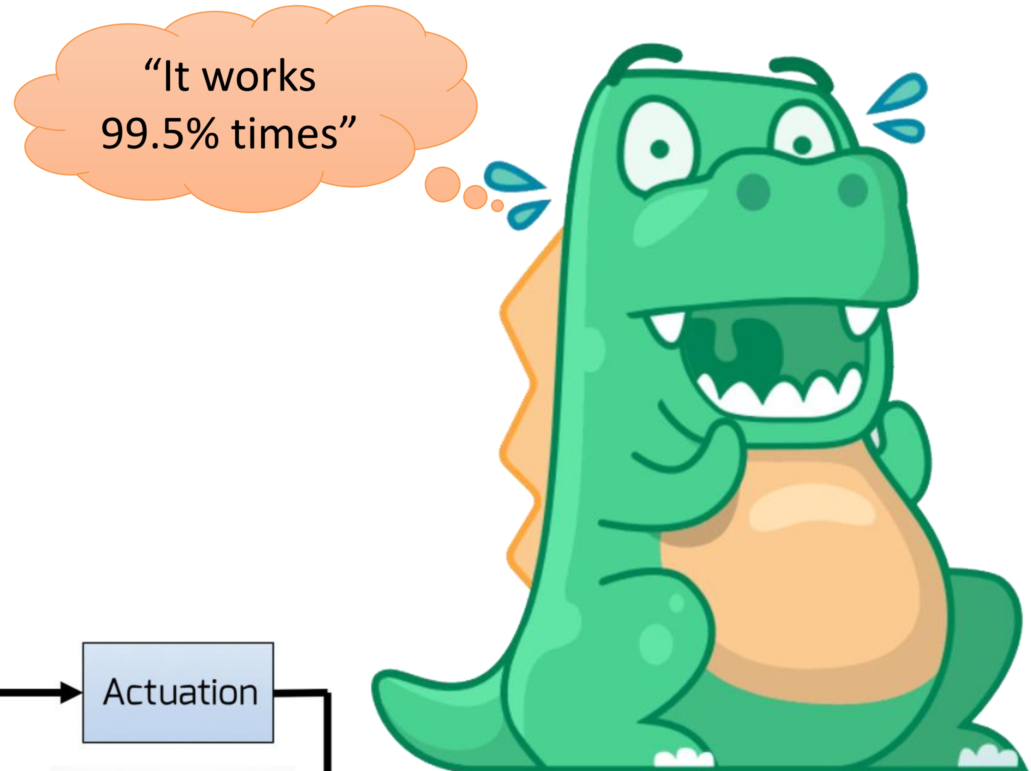
It's statistics, baby!

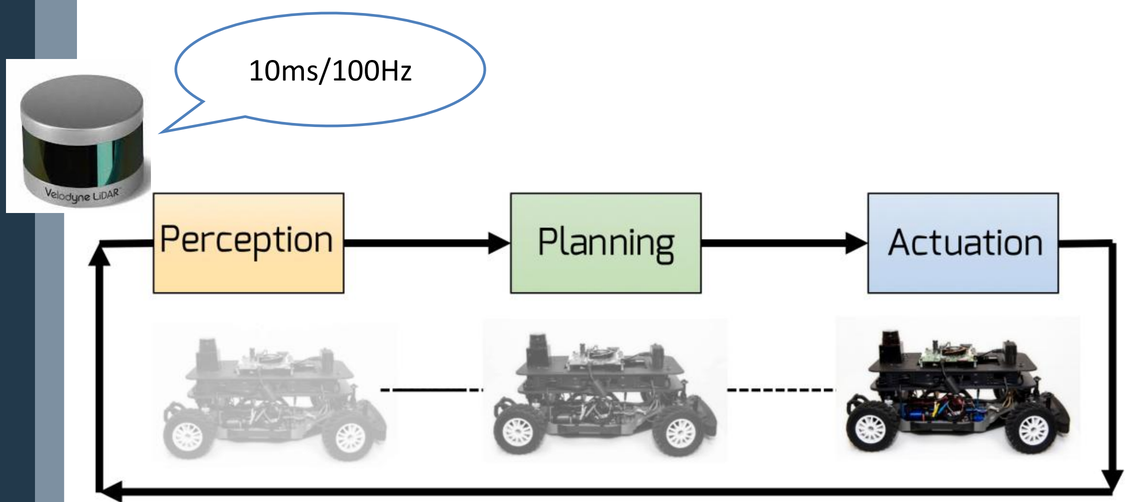


“I conti della serva”

Fails once every 200 times

=> Fails every 2 seconds





AI components monitoring and explainability

AI is inherently non-analyzable

› ..or, at least, it's very difficult to understand "what's in one's mind"

Still, we might need to monitor while running

› AI must be "explainable" in the sense that I need some indicators to measure its performance

› Trivial solution, analyzing how subset of the model (layers, or neurons) behave during training

› Employing redundancy at system level (e.g., voting systems)

› Monitor the Operational Design Domain, i.e., if the context stays within given parameters

  – "When it rains, my detection accuracy drops from 98% to 85%"

AI for design scalable systems

Areas

- Requirements collection
- System design & integration
- Coding
- Testing
- Team management

Requirements collection

Trivial: you can use AI as tools

- Read.ai for meeting minutes
- To create powerpoint presentations and docs

(But this is simple! Can we do more?)

AI could directly interact with customer

- ...can it? ☺

...but...

- AI can predict what customers want
- (With statistically meaning accuracy)

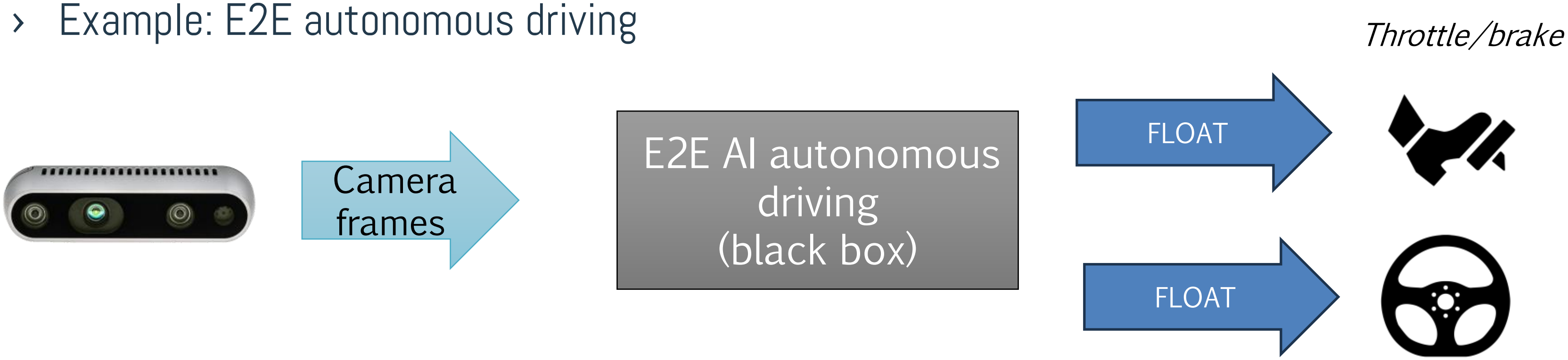
So, it can support you with useful information about your customer needs

- Based on previous customers
- “This guy is a tech guy, so he will provide you more tech information and you will need to elicit system-level information”

AI as software foundation

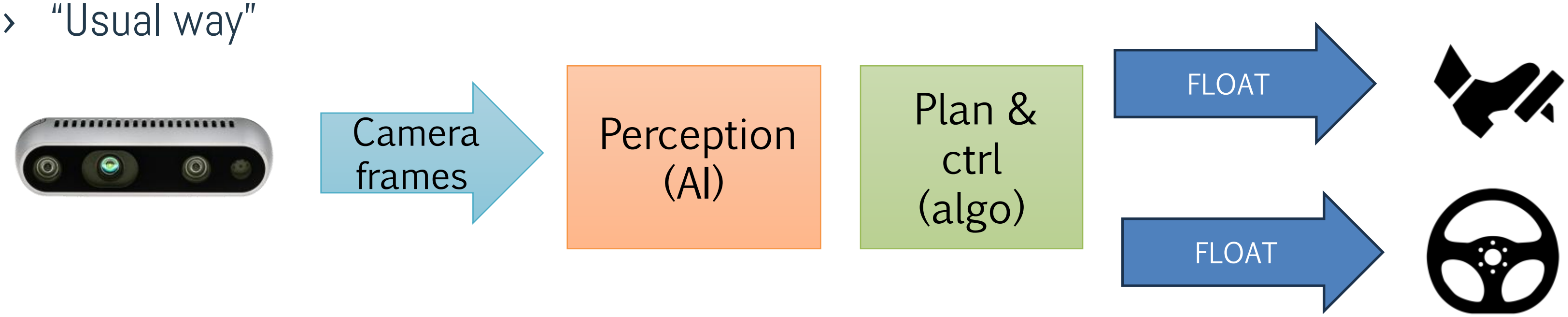
End-to-end approaches

- Your system is 100% based on AI. There are no algorithmic/deterministic components
- Also called “Behavioral cloning”
- Example: E2E autonomous driving



Integrating AI components

- “Usual way”



Testing

Straightforward from requirements analysis, we can generate tests automatically

- Several tools exists
- E.g., https://www.getxray.app/, https://www.testrail.com/
- Luckily, if a test fails due to wrong AI decision, we are forced to “look into it”, and refine

Remember: using AI means propagating mistakes and uncertainties from your development phases

- If requirements are not good (e.g., they don't cover all use cases) neither will your system
- So, the requirements collection must be well structured and oriented, e.g., to build a dataset
  - Check out the *Cucumber* language https://cucumber.io/
- “I your gym club, do you have customers of less than 18?”

AI and teams/processes

DORA - Software delivery performance metrics

Throughput

- #changes / time
- E.g., commits, LoC, push requests

VS.

Instability

- #issues / time
- E.g., “issues”, bugs (which now can be automatically detected” ☺)

Seven team profiles / clusters

1. Foundational challenges (10% overall)

› Problems in adopting technology

› Burnout and frictions

› Software is not stable

2. Legacy bottleneck (11% overall)

› Poor performance, still can deliver updates

› Burnout and frictions

› Software becomes stable after a number of iterations

Seven team profiles / clusters (cont'd)

3. Constrained by process (17% overall)

- Create low value, but still create it
- Burnout and frictions
- Software is stable – it's not about competences, but working environment/processes

4. High impact, low cadence (7% overall)

- Very good productivity
- Low frictions
- The environment is instable, harnessing long-term stability

5. Stable and methodical (15% overall)

- "Artisans": excellent productivity, however at low pace
- Healthy environment
- Software and environment are stable

Seven team profiles / clusters (cont'd)

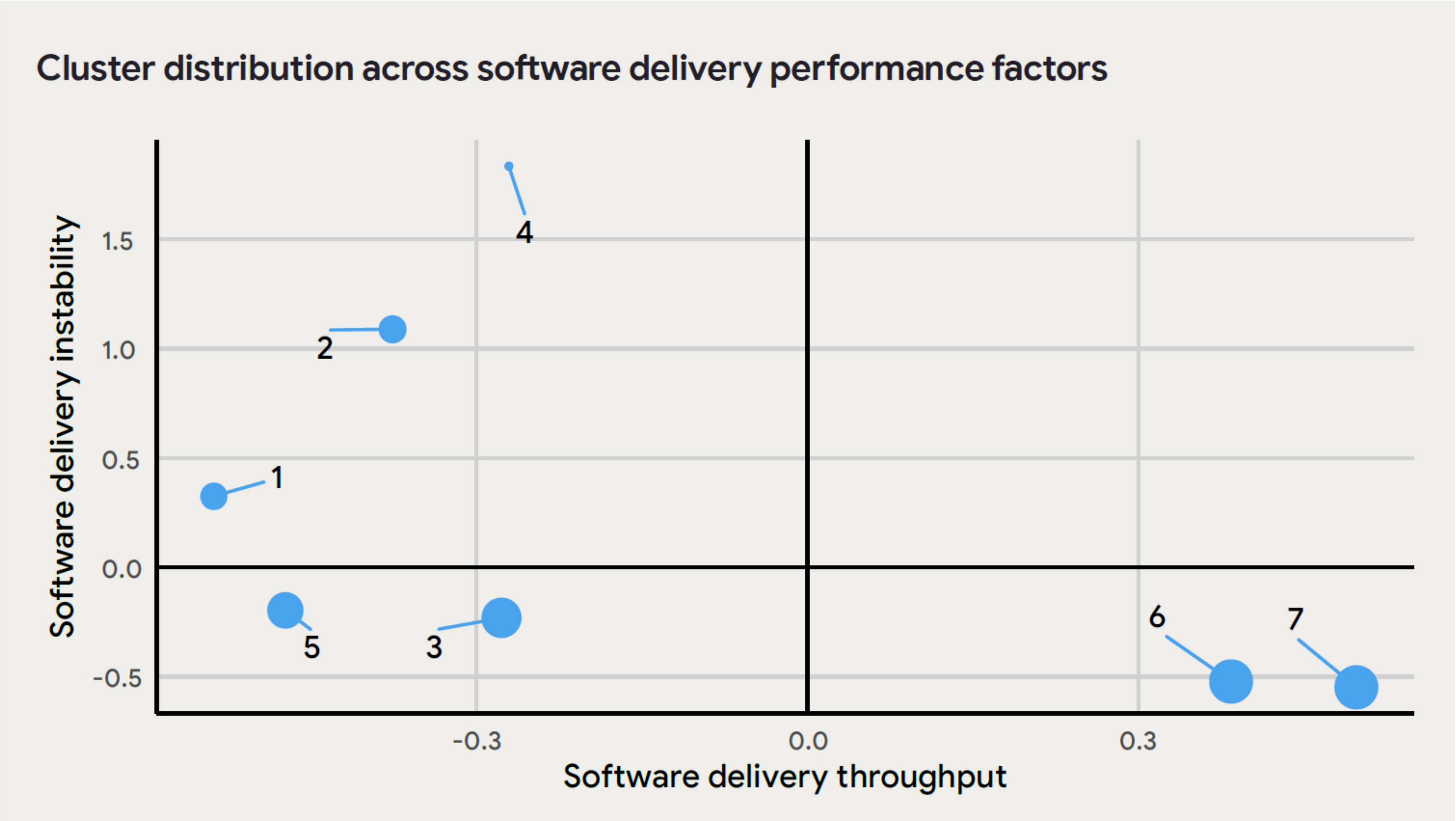
6. Pragmatic performance (20% overall)

- Good software, good pace
- Average burnout and frictions, but low engagement ("I work because you pay me but no more than this")
- Software and environment are stable

7. Harmonious high-achiever (20% overall)

- Good software, good pace
- Low burnout and frictions
- Software and environment are stable

At a glance



Cluster distribution across software delivery performance factors
Software delivery instability
1.5
1.0
0.5
0.0
-0.5
-0.3
0.0
0.3
Software delivery throughput
1
2
3
4
5
6
7

References

Course website

- http://hipert.unimore.it/people/paolob/pub/ProgSW/index.html

The DORA report: State of AI-assisted Software Development

- https://dora.dev/research/2025/dora-report/

My contacts

- paolo.burgio@unimore.it
- http://hipert.mat.unimore.it/people/paolob/